

TP6: Test Driven Development (TDD)

Fecha: 8/10/26

Modalidad: Clase virtual

Vamos a trabajar con el caso **TaxiMobile** para tener un ejemplo de cómo hacerlo

TP6:

- Entrega - Jueves 22 de octubre
- Se evalúa TDD y gestión Agile
- Hay que hacer una defensa de lo que hicimos

Sin desarrollar con TDD era muy probable que los programadores introdujera errores en su código y los pasen por alto, generando deuda técnica y retrabajo. Además, no es lo mismo resolver un error de forma temprana dentro de una misma iteración o etapa, que resolverla 5 meses después cuando ya escaló y su gravedad subió (mientras más tarde se encuentra, el costo **sube de manera exponencial**).

A partir de esto surge TDD. **TDD es una practica**



La idea de TDD es tomar un enfoque **proactivo para la resolución de errores**, resolviendo los errores en el momento en lugar de un enfoque reactivo como en la metodología tradicion

Las tres leyes de TDD



LAS 3 LEYES DE TDD

- "NO ESCRIBIRÁS UNA LÍNEA DE CÓDIGO DE PRODUCCIÓN SIN ANTES HABER ESCRITO UN TEST UNITARIO QUE FALLE"
- "NO ESCRIBAS MÁS CÓDIGO DE PRUEBA DEL NECESARIO PARA PROVOCAR UN FALLO. SI EL CÓDIGO NI SIQUIERA COMPILA, ESO YA CUENTA COMO FALLO."
- "NO ESCRIBIRÁS MÁS CÓDIGO DE PRODUCCIÓN QUE EL MÍNIMO INDISPENSABLE PARA QUE EL TEST PASE."

TDD se divide en 3 etapas que se desarrollan de forma cíclica:

1. **RED:** Escribimos una prueba que falle, definiendo el objetivo de la función o parte del sistema que quiero probar, definido de forma **específica y determinista** las **entradas** que tendrá y las **salidas** que devolverá. **Declaramos que es lo que debemos resolver**, luego vemos cómo lo implementamos.
Las pruebas pueden ser para caminos de **Fallas** o de **Éxito**.



RED

Trabajamos de manera declarativa

DECLARAMOS LA INTENCIÓN:
"QUIERO QUE AL PASARLE **ESTOS** DATOS, EL SISTEMA ME DEVUELVA **ESTE** RESULTADO"

```
def test_sumar_dos_numeros_EXITO():  
    //Precondiciones  
    calculadoraDePrueba = new Calculadora()  
    //Pasos  
    resultado = calculadoraDePrueba.sumar(a = 1,b = 2)  
    //Resultado  
    assert(resultado == 3)  
  
Class Calculadora:  
    def sumar(a,b):  
        return "No implementado" //Puede ser pass directamente
```

2. **GREEN:** Vamos a definir el **cómo vamos a pasar la prueba** implementando el **código mínimo necesario para pasar la prueba** (Sin darle bola a buenas prácticas, patrones, refactorización, mantenibilidad, etc). Hay múltiples patrones para desarrollar esto (como por ejemplo, el patrón **fake it** que básicamente consiste en hardcodear el

resultado que espera la prueba, es útil para funcionalidades muy grandes y complejas)

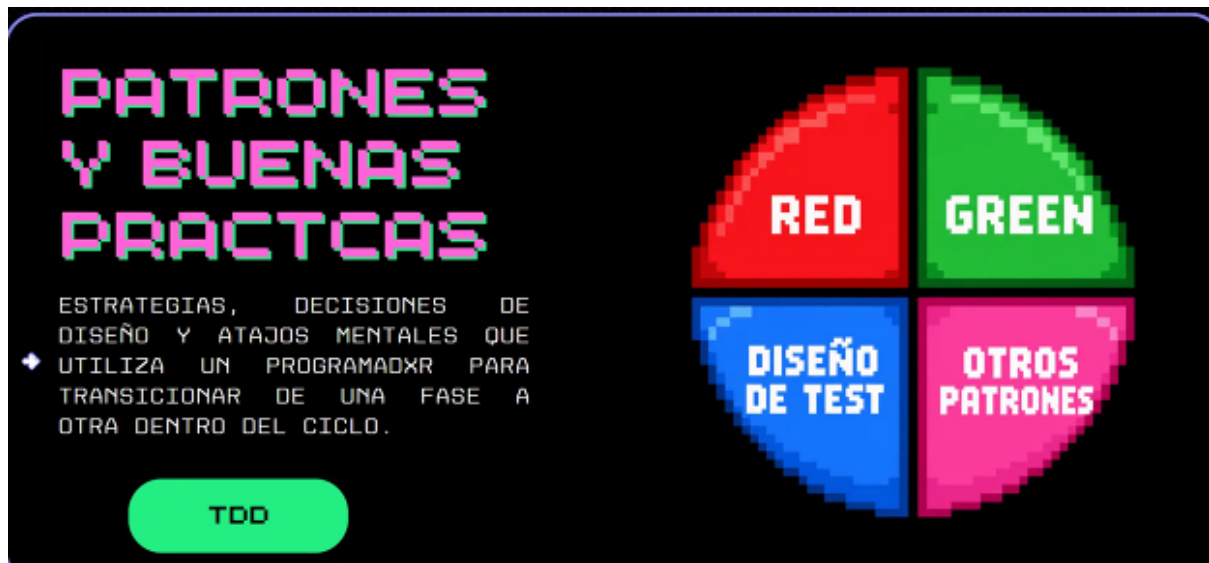


3. **REFACTOR:** En esta etapa **SI** tenemos que recordar y tener en cuenta las buenas prácticas, patrones y mantenibilidad. Refactorización de la **estructura interna de la función, sin modificar su comportamiento**.
4. **Volvemos a repetir el ciclo.** Volvemos a definir una prueba que falle (**RED**) para abarcar otro camino o posibilidad para la misma función, luego desarrollamos el código mínimo necesario para que la nueva prueba pase (**GREEN**) y finalmente la refactorizar aplicando buenas prácticas y mantenibilidad, SIN MODIFICAR EL COMPORTAMIENTO (**REFACTORING**). La idea de cada ciclo es ir mejorando el código y volviendo la solución más general para que pueda responder a todos los casos.
PARTIR DE PROBLEMAS PARTICULARES PARA TERMINAR RESOLVIENDO PROBLEMAS GENERALES



Patrones y buenas prácticas

Nos ayudan a definir cómo pasar de una etapa a otra, Los patrones se dividen en cada etapa



Patrones RED

Starter TEST

Elegir la prueba más trivial, el camino más fácil y obvio para poder comenzar con el ciclo. Nos permite destrabarse y pivotar a partir de allí a los caminos alternativos

One Step Test

Una vez que definimos un test, seguir con el siguiente en la línea de complejidad

Explanation Test

Enfocado en comprender el comportamiento de un sistema propio. Es una técnica para trabajar en equipo, donde se debate sobre el comportamiento esperado del sistema y a partir de las diferentes opciones se desarrollan los Test con el objetivo de validar o desmentir creencias o ideas. Nos ayuda a entender un comportamiento

Learning Test

Enfocado en comprender el comportamiento de código/sistemas de terceros. Para aprender sobre código que no es propio se desarrollan Test que permitan entender cómo funciona el sistema externo y como reacciona con el nuestro, una vez se aprende de ese sistema, se lo puede implementar de forma efectiva. **La idea es entender y aprender probandolo**

Another Test

Cuando nos alejamos del camino, frenamos, anotamos la problemática que queremos probar, y volvemos a centrarnos en la problemática y el Test que estamos desarrollando en

ese momento. básicamente, no irnos por las ramas, anotando las ideas que van surgiendo para no olvidarnos

Regression Test

Definir un Test que permita replicar los pasos que llevaron a un error, para asegurarnos que no vuelva a pasar. Claramente para esto necesitamos que durante el Testing exista trazabilidad de cómo se dio ese error para luego poder repetirlo y solucionarlo

Patrones GREEN

Fake it

hardcodear el retorno del código para que me de el resultado que la prueba necesita.

Triangulación

Armar varios casos de falla del sistema, para asegurarnos que el problema está efectivamente en nuestro código y no en una falla externa o particular

Obvious Implementation

Si la solución o funcionamiento del código es simple y obvio, podemos directamente desarrollar la implementación más óptima de manera que resuelva de forma correcta todos los casos generales

One to many

Enfocarnos primero en la “unidad” y a medida que realizamos iteraciones vamos agregando “colecciones o conjuntos” (todo esto referido a los parámetros)

Diseño de Test

Test list

Antes que nada, definir un listado de las funcionalidades que debemos testear

Isolated Test

Los patrones **DEBEN** ser independientes del resto. No se necesita de la ejecución de otro test para que el primero funcione

Test First

No escribir código antes de desarrollar las pruebas

Assert test

Desglosar desde el final hacia el principio (?)

Test Data

La información que debemos usar debe ser lo más sencillo y fácil de enter

Evident Data

La relación entre las entradas y las salidas debe ser la más obvia y sencilla posible



Caso práctico: Taxi Mobile (básicamente uber)

CASO TAXI MOBILE

Ver mapa de taxis

Como Administrador de la Central quiero ver la ubicación de todos los taxis de la central y si tienen viajes en curso para saber la disponibilidad actual

Criterios de Aceptación:

- Se muestran taxis en todos los estados (Verde: Libre, Amarillo: Solicitado, Rojo: Ocupado, Negro: Fuera de Servicio).
- Los taxis que se mostrarán deben tener habilitado el sistema de ubicación.
- Pasando el mouse sobre el ícono del auto deben poder visualizarse los datos del viaje si el taxi está ocupado (hora de inicio y costo) y los datos del pasajero (nombre, apellido y número de teléfono celular).
- Si el taxi está solicitado, sólo se visualizarán los datos del pasajero.
- Podrán utilizarse filtros por Barrio, por Estado y por chapa del taxi

Pruebas de Usuario

- Probar visualizar un conjunto de taxis libres (pasa)
- Probar visualizar un conjunto de taxis solicitados (pasa)
- Probar visualizar un conjunto de taxis fuera de servicio (pasa)
- Probar visualizar un conjunto de taxis ocupados (pasa)
- Probar visualizar un conjunto de taxis libres, solicitados, fuera de servicio y ocupados (pasa)
- Probar visualizar un taxi conectado al sistema por número de chapa (pasa)
- Probar visualizar en el mapa un taxi no conectado al sistema de ubicación (falla)
- Probar visualizar un conjunto de taxis en un Barrio inexistente (falla)

IXAT

The screenshot shows a yellow taxi with 'IXAT' on the roof, parked in front of a building.

- Primero definimos las **precondiciones** necesarias para realizar los pasos de la prueba. En este caso necesitamos loguear al usuario, registrar la ciudad de prueba y crear el taxi

- En la ejecución de un test se pueden probar varias cosas, no necesariamente una sola

RED



```
def test_mostrar_mapa_taxi_libres_en_barrio_exito ():
    //Precondiciones
    usuarios = [{user: "salvita", rol: "Administrador de la Central"}]
    loguear(salvita)
    configurar_ciudad_prueba("Córdoba Capital")
    taxis = [{"chapa": "ABC123", "estado": "Libre", "barrio": "Nueva Córdoba", "ubicacion": (10, 20)}]

    //Pasos
    mapa_mostrado = mostrar_mapa(filtros = ["barrio", "estado"], barrio = "General Paz", chapa = "", estado = "Libre")
    //Resultado
    assert all (mapa_mostrado.ciudad.barrio == "General Paz", "No existe tal barrio para esa ciudad ingresada")
    assert all(taxi.barrio == "General Paz" for taxi in mapa_mostrado.taxis, "No todos los taxis están dentro del area filtrada")
    assert all(taxi.estado == "Libre" for taxi in mapa_mostrado.taxis, "No todos los taxis están libres")
    assert all(taxi.icono == "verde" for taxi in mapa_mostrado.taxis, "No todos los taxis tienen el icono verde")

def mostrar_mapa(filtros, barrio, chapa, estado):
    pass
```

GREEN



```
def test_mostrar_mapa_taxi_libres_en_barrio_exito ():
    //Precondiciones
    usuarios = [{user: "salvita", rol: "Administrador de la Central"}]
    loguear(salvita)
    configurar_ciudad_prueba("Córdoba Capital")
    //carga los datos al sistema.
    taxis = [{"chapa": "ABC123", "estado": "Libre", "barrio": "General Paz", "ubicacion": (10, 20)}]

    //Pasos
    mapa_mostrado = mostrar_mapa(filtros = ["barrio", "estado"], barrio = "General Paz", chapa = "", estado = "Libre")
    //Resultado
    assert all (mapa_mostrado.ciudad.barrio == "General Paz", "No existe tal barrio para esa ciudad ingresada")
    assert all(taxi.barrio == "General Paz" for taxi in mapa_mostrado.taxis, "No todos los taxis están dentro del area filtrada")
    assert all(taxi.estado.nombre == "Libre" for taxi in mapa_mostrado.taxis, "No todos los taxis están libres")
    assert all(taxi.estado.icono == "verde" for taxi in mapa_mostrado.taxis, "No todos los taxis tienen el icono verde")

def mostrar_mapa(filtros, barrio, chapa, estado):
    return new Mapa([taxi={chapa: "ABC123", estado: {nombre: "Libre", icono: "verde"}, barrio: "General Paz"}])
```

REFACTOR

def mostrar_mapa(filtros: String[], barrio: Barrio, chapa: String, estado: Estado) -> Mapa:

```
    estado_libre = new Estado(  
        nombre = "Libre",  
        icono = "verde"  
    )
```

```
    barrio_general_paz = new Barrio(  
        nombre = "General Paz"  
    )
```

```
    taxi = new Taxi(  
        chapa = "ABC123",  
        estado = estado_libre,  
        barrio = barrio_general_paz  
    )
```

```
    taxis = [taxi]
```

```
    return new Mapa(taxis = taxis)
```



¿CÓMO SIGUE?

1. Volver a iniciar el ciclo.
2. Considerar el resto de casos de pruebas, añadiendo las alternativas planteadas en la user.
3. Refactorizar.

Hacer TDD sin refactorizar es simplemente escribir código malo con pruebas.



EJEMPLO FINAL

```
def mostrar_mapa(filtros: Filtros, barrio: Barrio, chapa: String, estado: Estado) -> Mapa:
```

```
// Caso de prueba taxis desconectados
```

```
if (filtros == 'Mostrar todos los taxis')
```

```
  taxis = buscarTaxiEnBarrio(barrio)
```

```
  (ubicación == None) //ubicación desconectada
```

```
  taxis = buscarTaxiDesconectado(ubicación)
```

```
// Caso de prueba mostrar taxis en distintos estados en un barrio
```

```
if (filtros == 'Mostrar taxis por estado y por barrio')
```

```
  taxis = buscarTaxiPorEstadoYPorBarrio(barrio, estado)
```

```
// Caso de prueba mostrar taxis por chapa
```

```
if (filtros == 'Mostrar taxis por chapa')
```

```
  taxis = buscarTaxiPorChapa(chapa)
```

```
// Caso de prueba mostrar taxis por barrio y datos de viaje
```

```
if (filtros == 'Mostrar taxis por barrio')
```

```
  taxisFiltrados = buscarTaxiEnBarrio(barrio)
```

```
//foreach simplificado
```

```
if (taxis[i].getEstado() == 'Solicitado' && filtros == 'Mostrar datos viaje')
```

```
  taxis =
```

```
    taxis[i].getChapa()
```

```
    taxis[i].getViajeActual().getPasajero().getNombre()
```

```
    taxis[i].getViajeActual().getPasajero().getTelefono()
```

```
    taxis[i].getViajeActual().getFechaYHora()
```

```
return new Mapa(taxis=taxis)
```

Pruebas de Usuario

- Probar visualizar un conjunto de taxis libres (pasa)
- Probar visualizar un conjunto de taxis solicitados (pasa)
- Probar visualizar un conjunto de taxis fuera de servicio (pasa)
- Probar visualizar un conjunto de taxis ocupados (pasa)
- Probar visualizar un conjunto de taxis libres, solicitados, fuera de servicio y ocupados (pasa)
- Probar visualizar un taxi conectado al sistema por número de chapa (pasa)
- Probar visualizar en el mapa un taxi no conectado al sistema de ubicación (falla)
- Probar visualizar un conjunto de taxis en un Barrio inexistente (falla)



Bibliografía

¡MUCHAS
GRACIAS!

- TEST DRIVEN DEVELOPMENT BY EXAMPLE. KENT BECK
- CLEAN CODE. ROBERT C. MARTIN
- ARTÍCULO EMBRACING TEST DRIVEN DEVELOPMENT. BRAD HUETT
- VIDEO: CODELYTV - APRENDE A TRIANGULAR EN TOD PARA PASAR DE ROJO A VERDE

Trabajo Práctico 6

Lo más importante es que de alguna forma ellos puedan ver que aplicamos de forma correcta la técnica

En el TP se evalúan dos cosas:

1. Cómo se aplica la técnica de desarrollo **aplicando requerimientos ágiles**. Por ejemplo: Las dudas que surjan durante el desarrollo necesitará un refinamiento con el PO (ERGO, HAY QUE PREGUNTAR SI O SI PIBÉ, porque no vas a aprobar sino y

además no vas a tener ni la más puta idea de como hacerlo). Las dudas los realizamos por mail, con una copia tanto para salva como para la valeria, hay que seguir las pautas de Email y hay que identificar al grupo

2. Claramente se va a evaluar **TDD**. Tenemos que dejar patente que aplicamos la técnica de desarrollo de TDD, para eso tenemos que crear una **suite de pruebas** y para dejar en claro el paso de pruebas podemos hacer **un commit por cada etapa que hacemos (para dejar documentado que lo estamos haciendo)**.

El commit no es necesariamente por cada funcionalidad o prueba individual, sino que podemos acumular pruebas de varias funcionalidades que están en una misma etapa y hacer un commit de ese conjunto de pruebas y así sucesivamente con todas las etapas.

Podemos hacer commits en etapas tempranas. Pero lo más interesante (**Y LO VERDADERAMENTE IMPORTANTE**) es hacer los commits de etapas más avanzadas o tardías y demostrar que hacemos TDD en esas etapas.

En conclusión: Aplica TDD en cada etapa de tu existencia en este TP

NO VAMOS A HACER PRUEBAS DEL FRONT. Enfoquémonos en pruebas de Backend. Se hacen **pruebas unitarias automatizadas**. Podemos hacer **pruebas de integración**, pero el foco son pruebas unitarias

Que hay que hacer:

- Programar una US de Eco Harmony de manera completa (esta en la consigna)
- Hay que desarrollar la US entera, osea, backend, frontend y base de datos
- Si no hay frontend, cagaste
- Vamos a tener que entregar un **Documento de reglas de estilo de código, como equipo tenemos que definir los estándares de cómo vamos a programar y cómo va a ser estructurado ese código** (convenios, librerías a usar, estándares, como pasar datos entre capas, nomenclatura, etc). Eso se **entrega por UV**. Podemos usar los convenios adoptados por el lenguaje o la comunidad

En resumen:

- Aplicar **TDD** en la US elegida
- Aplicar **Requerimientos Ágiles** en el desarrollo
- Definir el **Documento de reglas de estilo de código**